

Processeurs Multicœurs

César Fuguet

< cesar.fuguet@univ-grenoble-alpes.fr >

Inria, TIMA laboratory, Université Grenoble Alpes

Plan

1 Introduction

- Évolution des processeurs
- Processeurs multicœurs
- Défis

2 Organisations mémoire dans les processeurs multicœurs

- Processeur à mémoire non partagée distribuée
- Processeur à mémoire partagée centralisée (SMP)

- Processeur à mémoire partagée répartie (DSM)

3 Exemple de processeur multicœurs

4 Cohérence des mémoires cache

- Rappel sur les mémoires cache
- Problématique de la cohérence de caches
- Protocole avec espionnage (« snoop »)
- Limitations
- Passage à l'échelle
- Exercices

Plan

1 Introduction

- Évolution des processeurs
- Processeurs multicœurs
- Défis

2 Organisations mémoire dans les processeurs multicœurs

- Processeur à mémoire non partagée distribuée
- Processeur à mémoire partagée centralisée (SMP)

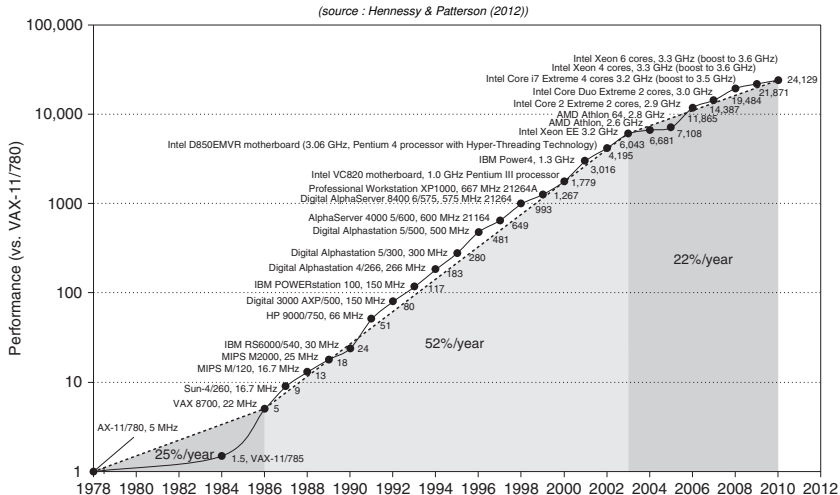
- Processeur à mémoire partagée répartie (DSM)

3 Exemple de processeur multicœurs

4 Cohérence des mémoires cache

- Rappel sur les mémoires cache
- Problématique de la cohérence de caches
- Protocole avec espionnage (« snoop »)
- Limitations
- Passage à l'échelle
- Exercices

Évolution des processeurs



« We are dedicating all of our future product development to multicore designs. We believe this is a key inflection point for the industry. »

Intel President Paul Otellini at Intel Developer Forum in 2005

Types de parallélisme

Classification

- Parallélisme au niveau des instructions (ILP)
→ Processeurs superscalaires
- *Parallélisme au niveau des tâches (TLP)*
→ Processeurs multicœurs.
- Parallélisme au niveau des données (DLP)
→ Processeurs vectoriels et processeurs graphiques (GPU)

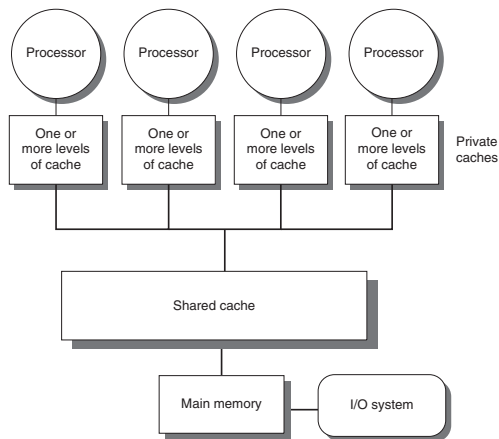
Selon Flynn

- SISD (Single Instruction Single Data)
→ ILP
- SIMD (Single Instruction Multiple Data)
→ DLP
- MISD (Multiple Instruction Single Data)
→ Processeur vectoriel systolique (?), pipeline (?), tolérance aux fautes.
- *MIMD (Multiple Instruction Multiple Data)*
→ TLP

Processeurs multicœurs

Définition

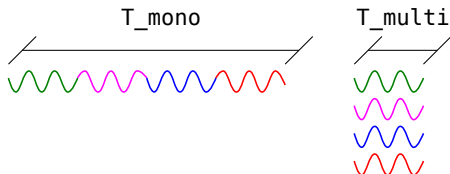
Processeur contenant plusieurs « cœurs » de calcul afin de permettre l'exécution de plusieurs tâches en parallèle (N cœurs $\rightarrow N$ tâches).



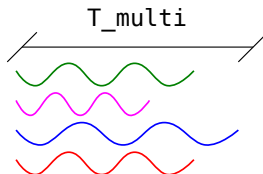
(source : Hennessy & Patterson (2012))

Sous-types de parallélismes dans les multicœurs

- Traitement parallèle au sein d'un même processus (P) -> *Parallel processing*.



- Traitement parallèle de plusieurs processus ($P_a, P_b, \dots, P_n$) -> *Request-level parallelism*.



T_{mono} : Temps d'exécution avec un seul cœur de calcul.

T_{multi} : Temps d'exécution avec plusieurs cœurs de calcul.

Processeurs multicœurs : défis

« Performance is now a programmer's burden. The La-Z-Boy programmer era of relying on hardware designers to make their programs go faster without lifting a finger is officially over »
(Hennessy & Patterson)

Principaux défis logiciels

- Le programmeur est le responsable de la parallélisation des applications.
- Gestion des ressources pour la part des systèmes d'exploitation (notamment l'ordonnancement des tâches et le placement des données dans la mémoire).
- Le gain obtenu par la parallélisation dépend du degré de parallélisme offert par l'application cible (**Loi d'Amdahl**).

Principaux défis matériels

- Latence/bande passante d'accès à la mémoire.
- Latence/bande passante du réseau de communication.
- Consommation énergétique.

Performances

- 1) En utilisant du traitement parallèle dans un même processus (P_a), quel est le temps minimum « théorique » d'exécution (T_{multi}) sur un processeur possédant N cœurs de calcul par rapport au temps d'exécution sur un seul cœur (T_{mono}) ?

Performances

- 1) En utilisant du traitement parallèle dans un même processus (P_a), quel est le temps minimum « théorique » d'exécution (T_{multi}) sur un processeur possédant N cœurs de calcul par rapport au temps d'exécution sur un seul cœur (T_{mono}) ?
- 2) Quand on exécute N processus indépendants ($P_a, P_b, \dots, P_n$) sur un processeur possédant un seul cœur de calcul, quel est le temps « théorique » d'exécution (T_{mono}), sachant que le temps d'exécution des processus est $T_a, T_b, \dots, T_n$ respectivement ?

Performances

- 1) En utilisant du traitement parallèle dans un même processus (P_a), quel est le temps minimum « théorique » d'exécution (T_{multi}) sur un processeur possédant N cœurs de calcul par rapport au temps d'exécution sur un seul cœur (T_{mono}) ?
- 2) Quand on exécute N processus indépendants ($P_a, P_b, \dots, P_n$) sur un processeur possédant un seul cœur de calcul, quel est le temps « théorique » d'exécution (T_{mono}), sachant que le temps d'exécution des processus est $T_a, T_b, \dots, T_n$ respectivement ?
- 3) Quand on exécute N processus indépendants sur un processeur possédant N cœurs de calcul, quel est le temps minimum « théorique » d'exécution (T_{multi}) ?

Performances

- 1) En utilisant du traitement parallèle dans un même processus (P_a), quel est le temps minimum « théorique » d'exécution (T_{multi}) sur un processeur possédant N cœurs de calcul par rapport au temps d'exécution sur un seul cœur (T_{mono}) ?
- 2) Quand on exécute N processus indépendants ($P_a, P_b, \dots, P_n$) sur un processeur possédant un seul cœur de calcul, quel est le temps « théorique » d'exécution (T_{mono}), sachant que le temps d'exécution des processus est $T_a, T_b, \dots, T_n$ respectivement ?
- 3) Quand on exécute N processus indépendants sur un processeur possédant N cœurs de calcul, quel est le temps minimum « théorique » d'exécution (T_{multi}) ?

Attention à la loi d'Amdahl ! Dans la pratique, les temps d'exécution sur un processeur multicœurs (T_{multi}) sont supérieurs à ceux ci-dessus !

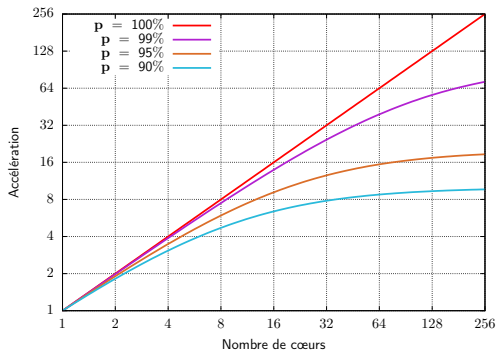
Loi d'Amdahl (I)

Énoncé

L'accélération théorique de l'exécution d'une tâche à travers sa parallélisation est toujours limitée par la partie non parallélisable.

$$f_{acc} = \frac{1}{(1-p) + \frac{p}{N}}$$

f_{acc}	facteur d'accélération
N	nombre de cœurs
p	partie parallèle du programme
$1 - p$	partie séquentielle du programme



Loi d'Amdahl (II)

- 1 Si l'on souhaite un facteur d'accélération d'au moins 75 sur un processeur contenant 100 cœurs de calcul, quelle fraction de l'application peut être séquentielle ?

Loi d'Amdahl (II)

- 1 Si l'on souhaite un facteur d'accélération d'au moins 75 sur un processeur contenant 100 cœurs de calcul, quelle fraction de l'application peut être séquentielle ?
- 2 La loi d'Amdahl peut aussi être utilisée dans des problèmes autres que le traitement parallèle. Elle permet de calculer le facteur d'amélioration de la performance d'un système lorsqu'une ou plusieurs composantes sont améliorées.

Exemple : Nous envisageons de faire une optimisation dans un processeur permettant d'exécuter 2 fois plus vite certaines instructions (p.e. multiplications/divisions). Si pour un programme donné, 20 % des instructions sont de ce type, quel est le facteur d'accélération ?

Plan

1 Introduction

- Évolution des processeurs
- Processeurs multicœurs
- Défis

2 Organisations mémoire dans les processeurs multicœurs

- Processeur à mémoire non partagée distribuée
- Processeur à mémoire partagée centralisée (SMP)

• Processeur à mémoire partagée répartie (DSM)

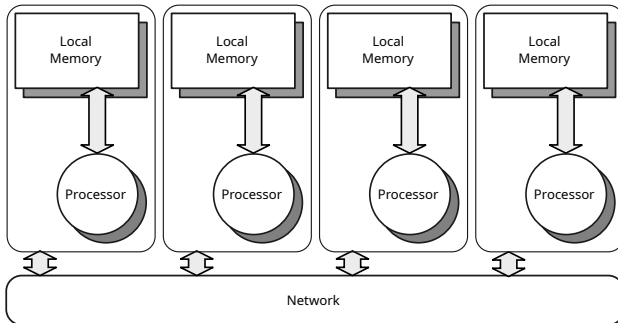
3 Exemple de processeur multicœurs

4 Cohérence des mémoires cache

- Rappel sur les mémoires cache
- Problématique de la cohérence de caches
- Protocole avec espionnage (« snoop »)
- Limitations
- Passage à l'échelle
- Exercices

Processeur à mémoire non partagée repartie

« Distributed Memory »

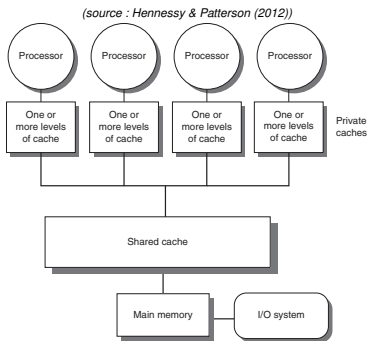


Vue du logiciel

- Chaque cœur a uniquement accès à sa mémoire locale.
- La communication avec les autres cœurs se fait par passage de messages.

Processeur à mémoire partagée centralisée

« Symmetric Multi-Processing (SMP) »

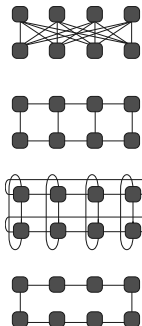
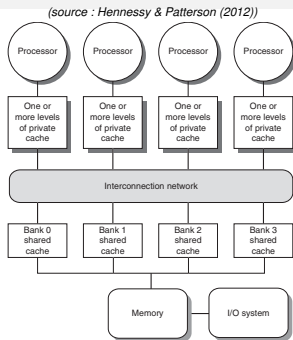


Vue du logiciel

- Tous les cœurs partagent la même mémoire.
- Les cœurs communiquent par des accès sur la mémoire partagée.
- Cette organisation est aussi appelée UMA (« Uniform Memory Access »).

Processeur à mémoire partagée répartie

« Distributed, Shared Memory (DSM) »

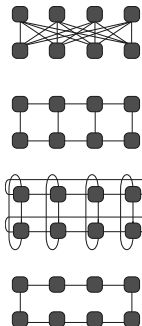
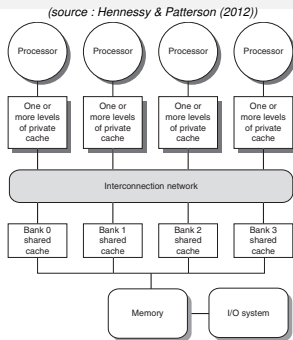


Vue du logiciel

- Tous les cœurs partagent les différents bancs mémoire.
- Les cœurs communiquent par des accès sur la mémoire partagée.
- Cette organisation est aussi appelée NUMA (« Non-Uniform Memory Access »).

Processeur à mémoire partagée répartie

« Distributed, Shared Memory (DSM) »



Complexité accrue pour le logiciel du à l'effet « NUMA » !

Le programmeur et/ou le système d'exploitation doivent non seulement distribuer les tâches dans les cœurs disponibles, mais aussi placer les données proches des cœurs les utilisant.

Plan

1 Introduction

- Évolution des processeurs
- Processeurs multicœurs
- Défis

2 Organisations mémoire dans les processeurs multicœurs

- Processeur à mémoire non partagée distribuée
- Processeur à mémoire partagée centralisée (SMP)

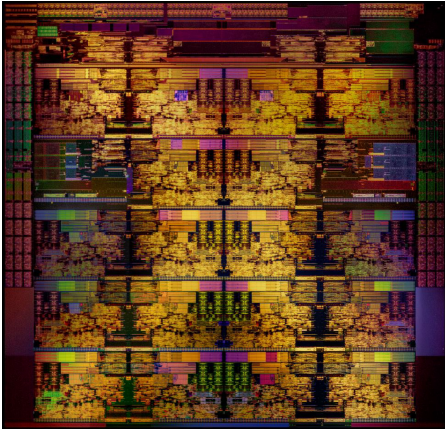
- Processeur à mémoire partagée répartie (DSM)

3 Exemple de processeur multicœurs

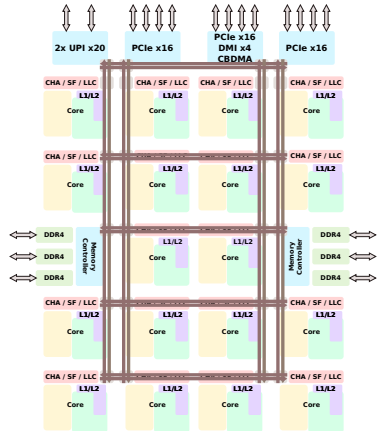
4 Cohérence des mémoires cache

- Rappel sur les mémoires cache
- Problématique de la cohérence de caches
- Protocole avec espionnage (« snoop »)
- Limitations
- Passage à l'échelle
- Exercices

Architecture Skylake d'Intel (version serveur)



(wikichip, skylake_(server))



Quelques chiffres

- 14 nm process
- # de transistors $\approx 8.000.000.000$ (estimé)
- 13 metal layers
- Surface du circuit $\approx 485mm^2$ (estimée)

Plan

1 Introduction

- Évolution des processeurs
- Processeurs multicœurs
- Défis

2 Organisations mémoire dans les processeurs multicœurs

- Processeur à mémoire non partagée distribuée
- Processeur à mémoire partagée centralisée (SMP)

- Processeur à mémoire partagée répartie (DSM)

3 Exemple de processeur multicœurs

4 Cohérence des mémoires cache

- Rappel sur les mémoires cache
- Problématique de la cohérence de caches
- Protocole avec espionnage (« snoop »)
- Limitations
- Passage à l'échelle
- Exercices

Les mémoires cache

Objectif

- Réduire le temps d'accès moyen aux données (latence).
- Réduire les besoins de bande passante mémoire.

Les caches s'appuient sur les principes suivants :

- **Localité temporelle** :

Une donnée qui vient d'être accédée sera probablement accédée dans un futur proche.

→ **Les mémoires cache copient au plus proche des cœurs les données récemment accédées.**

- **Localité spatiale** :

Les données proches en mémoire à une donnée qui vient d'être accédée seront aussi probablement accédée dans un futur proche.

→ **Les mémoires cache copient au plus proche des cœurs des blocs de données contenant la donnée accédée plus des données proches en mémoire.**

Les mémoires cache : terminologie et politique d'écriture

Terminologie

- **Succès de cache (« cache hit »)** :
Lorsque la donnée accédée se trouve dans la mémoire cache.
- **Échec de cache (« cache miss »)** :
Lorsque la donnée accédée ne se trouve pas dans la mémoire cache.
- **Ligne de cache (« cache block »)** :
Bloc de données contiguës en mémoire. Les caches stockent des blocs de données et les transferts entre les différents niveaux de cache et la mémoire se font par bloc.

Politiques d'écriture

- **Écriture immédiate (« write-through »)** :
Les écritures sont faites à la fois dans le cache (si succès) et dans le niveau suivant (inférieur) de la hiérarchie mémoire.
- **Écriture différée (« write-back »)** :
Si l'écriture dans le cache est un succès, uniquement la ligne à ce niveau est modifiée. Le contenu de la même ligne au niveau inférieur devient donc obsolète. Dans ce cas, la ligne de cache est dite modifiée ou « dirty ».

Cohérence des mémoires cache (I)

Problème de la cohérence de cache

- Lorsqu'un cœur accède à une adresse mémoire, la ligne de cache correspondante est copiée dans sa mémoire cache privée.
- Quand plusieurs cœurs accèdent à la même ligne de cache, chacun possède une copie dans sa mémoire cache privée. Si l'un des cœurs modifie une donnée de cette ligne, les autres cœurs auront une vue « *incohérente* » ou obsolète de cette donnée.

Types d'obsolescence :

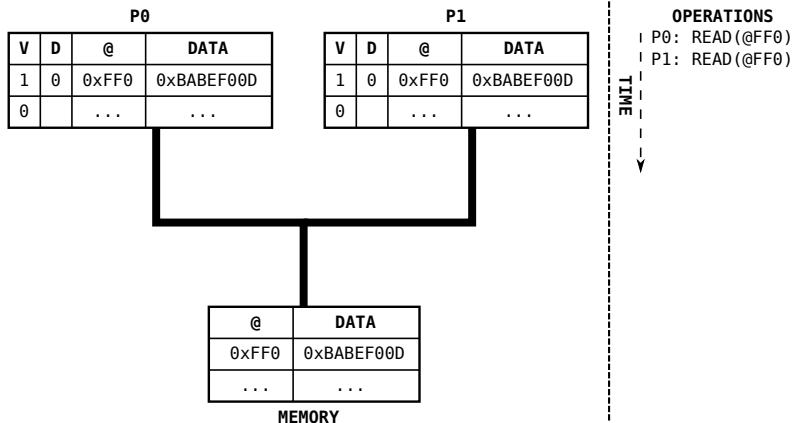
- *Obsolescence des caches* :

Des mémoires caches possèdent une vue obsolète d'une ligne de cache qui a été modifiée dans un autre cache. Existe avec les politiques « write-through » et « write-back ».

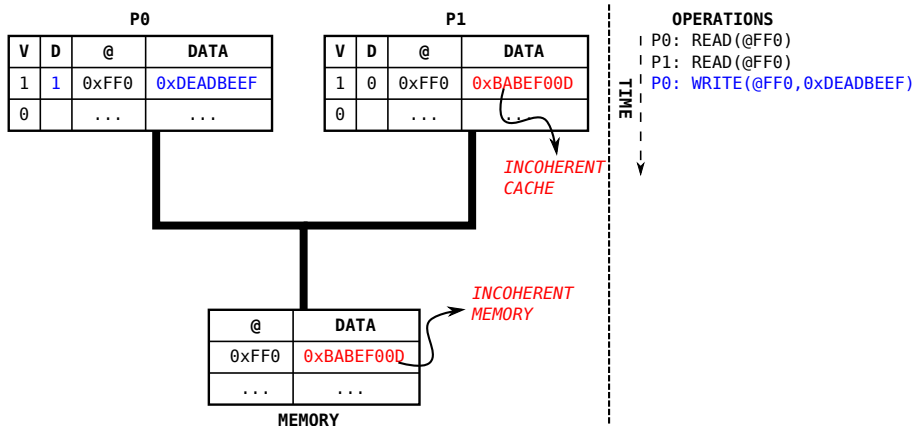
- *Obsolescence de la mémoire* :

La mémoire contient une vue obsolète d'une ligne de cache car elle a été modifiée dans une mémoire cache. Existe avec la politique « write-back ».

Cohérence des mémoires cache (II)



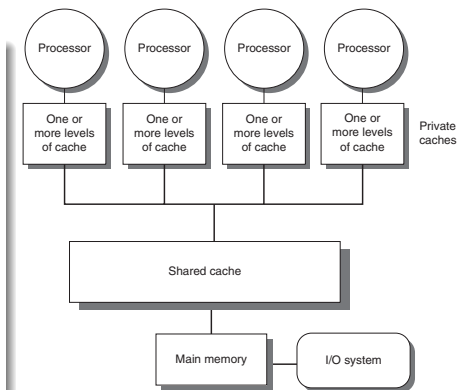
Cohérence des mémoires cache (III)



Protocole de cohérence avec espionnage (« snoop »)

Principes

- Toutes les mémoires cache maintiennent l'état de partage pour chaque ligne de cache qu'ils possèdent.
- Toutes les mémoires cache « espionnent » en permanence le réseau afin de déterminer si l'une des lignes dont ils ont une copie est la cible d'une requête.
- Le réseau supporte la diffusion (« broadcast ») d'une requête à toutes les mémoires cache du système.



(source : Hennessy & Patterson (2012))

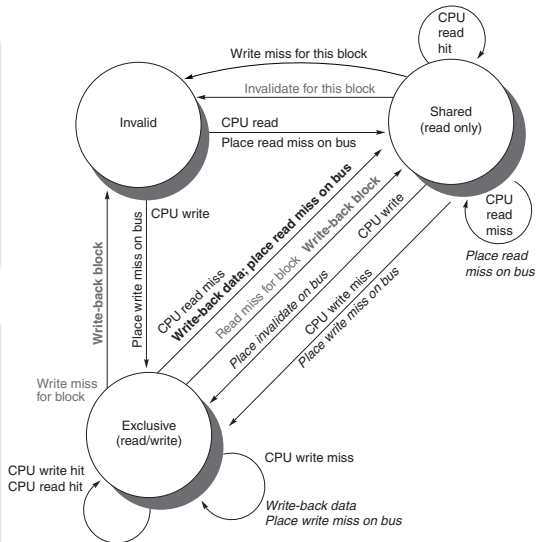
Protocole de cohérence MSI

Principes

- Protocole de type « **write invalidate** » (invalidation en écriture) : toutes les copies d'une ligne qui sera modifiée sont invalidées afin de garantir l'accès exclusif à cette ligne.
- Le protocole illustré ici s'appuie sur des caches utilisant une politique d'écriture différée (« write-back »).

États d'une ligne de cache

- **Modified/Exclusive** :
Ligne de cache modifiée et donc exclusive (une seule copie).
- **Shared** :
Ligne de cache partagée.
- **Invalid** :
Ligne de cache non-valide.



(source : Hennessy & Patterson (2012))

Protocole MSI : Exemple

P0 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	I	0x400	0x007D	0xF00D
1	M	0x414	0xA595	0x8282
2	S	0x408	0xBEEF	0xC07D
3	I	0x40C	0x3310	0x5278

P1 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	S	0x410	0x0908	0xCAFE
1	I	0x404	0x1023	0x4452
2	S	0x408	0xBEEF	0xC07D
3	M	0x40C	0x3310	0x50D1

OPERATIONS

TIME
↓

TRANSACTIONS (BUS)

@	DATA	
...	...	
0x400	0xBABE	0xF00D
0x404	0x1023	0x7878
0x408	0xBEEF	0xC07D
0x40C	0x3310	0x5278
0x410	0x0908	0xCAFE
0x414	0xAAAA	0x5555
...	...	

MEMORY

Protocole MSI : Exemple

P0 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	S	0x410	0x0908	0xCAFE
1	M	0x414	0xA595	0x8282
2	S	0x408	0xBEEF	0xC07D
3	I	0x40C	0x3310	0x5278

P1 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	S	0x410	0x0908	0xCAFE
1	I	0x404	0x1023	0x4452
2	S	0x408	0xBEEF	0xC07D
3	M	0x40C	0x3310	0x50D1

OPERATIONS

P0:R(@412)

#0

TIME
↓

TRANSACTIONS (BUS)

P0:RM(@410)

#0

@	DATA	
...	...	
0x400	0xBABE	0xF00D
0x404	0x1023	0x7878
0x408	0xBEEF	0xC07D
0x40C	0x3310	0x5278
0x410	0x0908	0xCAFE
0x414	0xAAAA	0x5555
...	...	

MEMORY

Protocole MSI : Exemple

P0 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	M	0x410	0xCAFE	0xCAFE
1	M	0x414	0xA595	0x8282
2	S	0x408	0xBEEF	0xC07D
3	I	0x40C	0x3310	0x5278

P1 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	I	0x410	0x0908	0xCAFE
1	I	0x404	0x1023	0x4452
2	S	0x408	0xBEEF	0xC07D
3	M	0x40C	0x3310	0x50D1

@	DATA
...	...
0x400	0xBABE 0xF00D
0x404	0x1023 0x7878
0x408	0xBEEF 0xC07D
0x40C	0x3310 0x5278
0x410	0x0908 0xCAFE
0x414	0xAAAA 0x5555
...	...

MEMORY

OPERATIONS

P0:R(@412) #0
 P0:W(@412,0xCAFE) #1

TIME
 ↓

TRANSACTIONS (BUS)

P0:RM(@410) #0
 P0:IV(@410) #1

Protocole MSI : Exemple

P0 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	I	0x410	0xCAFE	0xCAFE
1	M	0x414	0xA595	0x8282
2	S	0x408	0xBEEF	0xC07D
3	I	0x40C	0x3310	0x5278

P1 . CACHE

	STATE	@	DATA[1]	DATA[0]
0	M	0x410	0xCAFE	0x0000
1	I	0x404	0x1023	0x4452
2	S	0x408	0xBEEF	0xC07D
3	M	0x40C	0x3310	0x50D1

@	DATA	
...	...	
0x400	0xBABE	0xF00D
0x404	0x1023	0x7878
0x408	0xBEEF	0xC07D
0x40C	0x3310	0x5278
0x410	0xCAFE	0xCAFE
0x414	0xAAAA	0x5555
...	...	

MEMORY

OPERATIONS

| P0:R(@412) #0
 | P0:W(@412,0xCAFE) #1
 | P1:W(@410,0x0000) #2

TIME
↓

TRANSACTIONS (BUS)

P0:RM(@410) #0
 P0:IV(@410) #1
 P1:WM(@410),P0:WB(@410) #2

Limitations des protocoles avec espionnage

Limitations

Le principal problème des protocoles avec espionnage est leur passage à l'échelle.

Lorsque le nombre de cœurs dans le système augmente :

- La contention sur le réseau de communication devient très gênante à l'augmentation de la performance. Ceci est dû au fait que les différents messages doivent parvenir à toutes les mémoires cache du système,
- Les caches sont souvent occupés à traiter les messages venant du réseau ce qui les empêche de servir leur propre cœur dégradant ainsi les performances.
- L'efficacité énergétique est fortement dégradée car les messages sur le réseau, en plus de créer la contention, engendrent une surconsommation énergétique.

En général, le nombre maximum de cœurs dans un système s'appuyant uniquement sur un protocole de cohérence avec espionnage est 8.

Passage à l'échelle

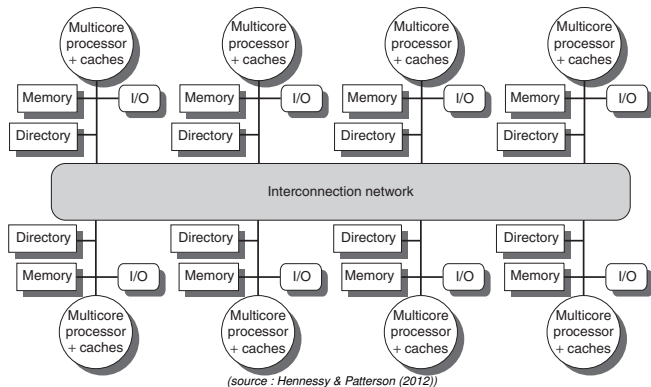
Plusieurs techniques sont utilisées afin de permettre le passage à l'échelle des protocoles de cohérence de cache.

La plupart des protocoles visant à supporter plusieurs dizaines, voire centaines des cœurs s'appuient sur un répertoire pour remplacer la technique de l'espionnage.

Répertoire

- Structure contenant, pour chaque ligne de cache, la liste de caches contenant une copie.
- Cette structure est implémentée au niveau de la mémoire ou de la mémoire cache partagée.
- Fonctionnement : Lorsqu'un cache fait une demande sur une ligne de cache, au lieu d'envoyer le message correspondant à tous les caches du système, il émet sa demande au répertoire qui ensuite propage celle-ci **uniquement** aux mémoires cache ayant une copie.

Exemple d'architecture implémentant des répertoires pour la cohérence de caches



- Lorsque le système implémente une mémoire répartie, le répertoire est lui aussi réparti.
- Pour que cette technique soit efficace, le réseau doit être hiérarchique (pas un bus).
- Il est possible d'implémenter de protocoles hybrides : connecter plusieurs multicœurs ensemble, chaque multicœurs implémente de l'espionnage localement mais un répertoire pour les échanges distants.

Exercices (I)

Exercice 1

En partant du dernier état du système (contenu des caches et de la mémoire) dans l'exemple du protocole MSI, exécuter pas à pas les opérations décrites ci-après et à chaque étape :

- 1) écrire le nouvel état des caches et de la mémoire ;
- 2) écrire les transactions déclenchées par ces opérations sur le bus.

P0 : R(@400)

P1 : R(@414)

P1 : W(@414, 0x4141)

P0 : R(@414)

Exercices (II)

Exercice 2

En partant de l'automate d'états du protocole MSI montré préalablement, faire les exercices ci-après. Pour chaque exercice, dessiner l'automate du protocole après les modifications que vous proposez.

- 1 Quelles modifications feriez-vous au protocole afin de l'adapter à des caches implémentant une politique d'écriture immédiate (« write-through ») ?
- 2 Quelles modifications feriez-vous au protocole afin de remplacer la politique « write-invalidate » par une politique « write-update » (écriture puis mise à jour) ?

Remarque : avec la politique « write-update », en cas d'écriture, au lieu d'invalider les copies des autres mémoires cache, le cache faisant l'écriture diffuse la nouvelle valeur à tous les autres.

Références

- Hennessy & Patterson (2012). Computer architecture : a quantitative approach. Elsevier.
- Borkar, S. (2007, June). Thousand core chips : a technology perspective. In Proceedings of the 44th annual Design Automation Conference (pp. 746-749). ACM.
- [https://en.wikichip.org/wiki/intel/microarchitectures/skylake_\(server\)](https://en.wikichip.org/wiki/intel/microarchitectures/skylake_(server))

Merci

Questions ?

5 Appendix